

AS SOLUÇÕES ESTÃO NA FORMA NR (NÃO RECURSIVA) SEGUIDAS DA SOLUÇÃO RECURSIVA (R)

1. Calcular a soma dos elementos de uma lista numérica.

```
def soma_elems_NR(lst):  
    soma = 0  
    for i in range(len(lst)):  
        soma = soma + lst[i]  
    return soma  
# fim soma_elems_NR
```

1. Calcular a soma dos elementos de uma lista numérica.

```
def soma_elems_R(lst):  
    if lst==[]:  
        return 0  
    else:  
        return lst[0] + soma_elems_R(lst[1:])  
# fim soma_elems_NR
```

#####

*# 2. Calcular a potencia de um número qualquer a elevado a um expoente inteiro positivo b:
a elevado a b.*

```
def potenciaNR(a,b):  
    if b == 0:  
        return 1  
    else:  
        pot = 1  
        for i in range(b):  
            pot = pot * a  
        return pot
```

```
def potenciaR(a,b):  
    if b == 0:  
        return 1  
    elif a == 0:  
        return 0  
    else:  
        return a * potenciaR(a,b-1)
```

#####

3. Inverter uma string de entrada.

```
def inverteStrNR(s):  
    si = ''  
    for c in s:  
        si = c + si  
    return si  
# fim inverteStrNR
```

```
def inverteStrR(s):  
    if s == '':  
        return s  
    else:  
        return inverteStrR(s[1:]) + s[0]  
# fim inverteStrR
```

#####

4. Testar se uma string de entrada é um palíndromo. Retorna True caso seja, False caso não seja um palíndromo.

```
def palindromoNR(s):  
    sp = ''  
    i = len(s) - 1  
    while i >= 0:  
        sp = sp + s[i]  
        i = i - 1  
    # while..  
    return s == sp  
# fim palindromoNR
```

```

def palindromoR(s):
    if s == '':
        return True
    elif len(s) == 1:
        return True
    else:
        return (s[0] == s[-1]) and palindromoR(s[1:-1])
# fim palindromoR

#####

# 5. Calcular o produto de 2 números, x e y.
def mult_NR(x,y):
    soma = 0
    for i in range(x):
        soma = soma + y
    return soma
# fim mult_NR

# 5. Calcular o produto de 2 números, x e y.
def mult_R(x,y):
    if x == 0:
        return 0
    else:
        return y + mult_R(x-1,y)
# fim mult_R

#####

# 6. Calcular a divisão inteira de 2 números, x e y. (pesquise o conceito de divisão
inteira)
def div_NR(x,y):
    cont = 0
    while x >= y:
        x = x - y
        cont = cont + 1
    return cont
# fim div_NR

def div_R(x,y):
    if x < y:
        return 0
    else:
        return 1 + div_R(x-y,y)
# fim div_R

#####

# 7. Pesquisar a existência do elemento e na lista L.
def pesquisaLstNR(elem,lst):
    i = 0
    while (i < len(lst)) and (elem != lst[i]):
        i = i + 1
    return i < len(lst)
# fim pesquisaLstNR

def pesquisaLstR(elem,lst):
    if lst == []:
        return False
    elif elem == lst[0]:
        return True
    else:
        return pesquisaLstR(elem,lst[1:])
# fim pesquisaLstR

#####

# 8. Calcular o maior valor de uma lista de números fornecida como entrada.

```

```

def maiorNR(lst):
    if lst == []:
        return None
    else:
        biggest = lst[0]
        i = 0
        while i < len(lst):
            if lst[i] > biggest:
                biggest = lst[i]
            i = i + 1
        return biggest
# fim maiorNR

def maiorR(lst):
    if lst == []:
        return None
    elif len(lst) == 1:
        return lst[0]
    else:
        # 0 maior de todos é o primeiro, se ele for o maior que o maior do restante da
        # lista.
        if lst[0] >= maiorR(lst[1:]):
            return lst[0]
        # Caso contrário, o maior é o maior dentre todo o restante da lista.
        else:
            return maiorR(lst[1:])
# fim maiorR

#####

# 9. Calcular o menor valor de uma lista de números fornecida como entrada.
def menorNR(lst):
    if lst == []:
        return None
    elif len(lst) == 1:
        return lst[0]
    else:
        # 0 menor de todos é o primeiro, se ele for o menor que o menor do restante da
        # lista.
        if lst[0] < menorR(lst[1:]):
            return lst[0]
        # Caso contrário, o menor é o menor dentre todo o restante da lista.
        else:
            return menorR(lst[1:])
# fim menorNR

def menorR(lst):
    if lst == []:
        return None
    else:
        menordetodos = lst[0]
        i = 0
        while i < len(lst):
            if lst[i] < menordetodos:
                menordetodos = lst[i]
            i = i + 1
        return menordetodos
# fim menorR

#####

# 10. Calcular a raiz quadrada de um número n com tolerância máxima t. (pesquise a
# definição de raiz quadrada)
# x: número que se quer saber a raiz, x0: aproximação inicial, e: tolerância.
def raizqNR(x,x0,e):
    while abs(x0**2 - x) > e:
        x0 = (x0**2 + x)/(2 * x0)
    return x0
# fim raizqNR

```

```

# 9. Calcular a raiz quadrada de um número n com tolerância máxima t. (pesquise a definição de raiz quadrada)
# x: número que se quer saber a raiz, x0: aproximação inicial, e: tolerância.
def raizqR(x,x0,e):
    if abs(x0**2 - x) <= e:
        return x0
    else:
        x0 = (x0**2 + x)/(2 * x0)
        return raizqR(x,x0,e)
# fim raizqR

#####

# 11. Verificar se um número inteiro N é primo. Retorna True caso seja primo, False caso contrário. Dica: comece a divisão pelo número sobre 2, crie uma função que receba o número e o primeiro divisor.
# Na chamada da função, div inicia com num//2: Ex. isprimo(17,17//2)
def isprimoNR(num, div):
    if num == 1:
        return False

    for i in range(div,1,-1):
        if num % i == 0:
            return False

    return True
# fim isprimoNR

# Na chamada da função, div inicia com num//2: Ex. isprimo(17,17//2)
def isprimoR(num, div):
    if num == 1:
        return False

    if div == 1:
        return True
    else:
        if num % div == 0:
            return False
        else:
            return isprimoR(num, div - 1)
# fim isprimoR

#####

# 12. É possível construir uma função recursiva para converter um valor em base dez para binário? Tente construir esta função a partir do algoritmo clássico de conversão decimal binário.
def dec2binNR(n):
    nbin = 0
    pot10 = 0

    while n != 0:
        q = n//2
        r = n%2
        nbin = nbin + r*(10**pot10)
        pot10 = pot10 + 1
        n = q
    # while
    return nbin
# fim dec2binNR

def dec2binR(n):
    if n == 0:
        return 0
    else:
        return n%2 + (dec2binR(n//2) * 10)
# fim dec2binR

```